

S5: Algorithmic Optimizations for Generic Nonplanar 4-Axis Slicing

JACOB NAVARRETE and DR. TED FORRINGER, University of North Georgia, USA

Nonplanar fused-deposition modeling (FDM) on multi-axis printers leverages rotational axes to fabricate contoured surfaces without supports. Traditional planar slicers designed for Cartesian kinematics can not interpret rotational motion to conform to organic surface geometry, which increases material waste and post-processing cost. The reference *S4_Slicer* framework addresses this through tetrahedral mesh construction, overhang-aware rotation-fields, mesh deformation, and synthesizing G-code into a custom machine coordinate space. However, its implementation relies on per-cell Python iteration, NetworkX graph traversals, and general-purpose nonlinear least-squares solvers applied to problems that are exactly quadratic. The result is severe scalability bottlenecks: $O(n)$ Python loops, Jacobian rebuilds inside trust-region iterations, and sequential per-point barycentric solves dominate wall time. This paper presents *S5_Slicer*: a suite of nine algorithmic optimizations that reformulate each pipeline stage as either a sparse linear system, a quadratic program, or a batched generalized universal function (gufunc). We (i) recast neighbor-adjacency construction as a COO sparse-matrix scatter; (ii) vectorize Dijkstra predecessor tracing via fixed-point propagation over boolean masks; (iii) batch per-cell SVD plane-fits through a single Einstein-summation-plus-LAPACK call; (iv) replace iterative Trust-Region-Reflective (TRF) smoothing with a one-shot UMFPAK factorization of the graph-Laplacian system; (v) implementation of three independent LSQR problems concurrently under GIL-releasing CSR matvec; and (vi)–(ix) vectorize barycentric solves, vertex-rotation recovery, volume scaling, and validity masking. For a representative 50,000-cell mesh these changes reduce dominant stage wall times by up to four orders of magnitude while preserving solution correctness, increasing slicing efficiency while reducing post-process manufacturing costs.

Additional Key Words and Phrases: nonplanar slicing, multi-axis 3D printing, sparse linear algebra, graph Laplacian, tetrahedral mesh, barycentric coordinates, sparse least-squares, parallel computing

ACM Reference Format:

Jacob Navarrete and Dr. Ted Forringer. 2026. S5: Algorithmic Optimizations for Generic Nonplanar 4-Axis Slicing. 0, 0, Article 0 (2026), 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Cartesian and CoreXY gantry printers deposit material in horizontal layers of constant Z -height through planar FDM. This approach introduces two limitations. First, surfaces that face downward relative to the print direction require support material, increases post-processing time and wastes filament. Second, the resulting horizontal layer lines act as stress concentrator, weakening parts along the build direction by between 20% and 60% relative to injection-molded counterparts [2].

Authors' Contact Information: Jacob Navarrete, jacob@jacobandres.com; Dr. Ted Forringer, University of North Georgia, Gainesville, Georgia, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/0-ART0

<https://doi.org/XXXXXXX.XXXXXXX>

Nonplanar FDM addresses both limitations by tilting the nozzle or rotating the build plate so depositing toolpaths follow curved surfaces. On a 4-axis rotational machine with axes C (rotation about $\hat{z}$), X (radial), Z (vertical), and B (nozzle tilt), the tool can continuously reorient with respect to the surface geometry, eliminating overhangs and aligning deposition roads with principal stress directions.

A nonplanar slicer must (i) analyze the input geometry to identify overhang regions, (ii) compute a spatially varying rotation field that maps each volumetric element to an appropriate nozzle tilt, (iii) deform the mesh so that a standard planar slicer (Cura) produces correct toolpaths in a warped coordinate space, and (iv) invert the deformation to recover physical machine coordinates. Each stage operates on tetrahedral meshes of tens of thousands of elements, so computational efficiency is critical.

The reference implementation *S4_Slicer* [3] adapts the general nonplanar framework to the Core-R-Theta class of 4-axis printers. It is functionally correct but relies heavily on Python-level iteration, NetworkX graph traversals, and iterative nonlinear least-squares solvers in hot paths. Profiling reveals three dominant bottlenecks:

- (1) $O(n)$ -per-cell Python while-loops for Dijkstra predecessor tracing and gradient fitting;
- (2) Trust-Region-Reflective (TRF) least-squares applied to objectives that are, mathematically, exactly quadratic;
- (3) Per-point sequential barycentric solves in the G-code reformation stage.

This paper makes the following contributions:

- A formal characterization of each pipeline stage as a sparse linear program, a quadratic program, or a batched gufunc, enabling the substitution of general-purpose solvers with direct and batched methods.
- Nine concrete algorithmic optimizations, each accompanied by a complexity analysis and the underlying mathematical derivation.
- A discussion of upgrade paths (CHOLMOD, PyPardiso, CuPy) for models exceeding 5×10^4 cells.

2 Background and Related Work

2.1 Nonplanar FDM Slicing

Ichida et al.[10] introduced curved-layer FDM for shell-like geometries. Gradient-based methods that compute per-voxel print directions were developed by Etienne et al.[6] and extended to full volumetric optimization on multi-axis kinematics by [7]. Industrial 5-axis approaches include Wu et al.[16] who formulate the direction field as a sparse linear system over a tetrahedral mesh, the same mathematical structure applied in Section 4.4.

The *S4_Slicer* [3] adapts these ideas to a rotational 4-axis kinematics where the build plate rotates about $\hat{z}$ (the C -axis) and the nozzle tilts (the B -axis), producing cylindrical rather than Cartesian toolpaths. Its pipeline consists of five stages: (1) tetrahedral mesh generation via TetGen [11]; (2) overhang analysis and rotation-field

initialization; (3) Laplacian-smoothed rotation-field optimization; (4) mesh deformation; (5) barycentric G-code remapping.

2.2 Sparse Linear Algebra in Geometry Processing

The graph Laplacian of a mesh, $\mathcal{L} = \mathbf{D} - \mathbf{A}$, where $\mathbf{D}$ is the degree matrix and $\mathbf{A}$ is the adjacency matrix, appears in deformation [12], parameterization [8], and smoothing [13]. When the optimization objective is a sum of squared affine functions of the decision variables, its minimizer satisfies a linear system whose coefficient matrix is symmetric positive-definite and equal to a regularized Laplacian. Direct solvers such as UMFPACK [5] and CHOLMOD [4] exploit this sparsity to achieve $O(n^{1.5})$ fill-in complexity on planar graphs.

The observation driving this work is that several S4_Slicer stages previously implemented as iterative nonlinear optimizations are, in fact, *exactly quadratic* in their decision variables and therefore admit a one-shot direct sparse solve.

2.3 Vectorized Numerical Computing

NumPy's generalized universal function (*gufunc*) infrastructure allows many linear-algebra routines such as matrix solves, SVD, and determinants to be broadcast over leading batch dimensions [9]. When the same operation must be applied to n independent instances of small matrices, batched *gufuncs* eliminate Python dispatch overhead. Additionally, SciPy's CSR matrix-vector product releases the CPython Global Interpreter Lock [15], which admits thread-level parallelism for independent linear systems through the standard library's ThreadPoolExecutor.

3 Pipeline Overview

Let $\mathcal{M} = (\mathcal{V}, \mathcal{C})$ denote the tetrahedral mesh output by TetGen, where $\mathcal{V} \subset \mathbb{R}^3$ is the vertex set with $n_p = |\mathcal{V}|$ points and $\mathcal{C} \subset \mathcal{V}^4$ is the set of tetrahedra with $n_c = |\mathcal{C}|$ cells. For each cell $c \in \mathcal{C}$, let $\mathbf{v}_c = (v_{c,0}, v_{c,1}, v_{c,2}, v_{c,3}) \in \mathbb{R}^{4 \times 3}$ denote its vertex stack. The surface mesh $= (\mathcal{V}_S,)$ is extracted as the boundary of $\mathcal{C}$.

For each cell the pipeline computes the following quantities:

$\mathbf{n}_c \in \mathbb{R}^3$	outward normal of lowest- z face of c ,
$\theta_c \in [0, \pi]$	overhang angle, $\arccos(\mathbf{n}_c \cdot \hat{z})$,
$d_c \in \mathbb{R}_{\geq 0}$	geodesic distance from c to the bottom source set $\mathcal{B}_S$ via the heat method,
$r_c \in [-2\pi, 2\pi]$	scalar rotation field (nozzle B -axis),
$\mathbf{R}_c \in SO(3)$	per-cell rotation matrix about the tangent axis at c .

The tangential axis $\mathbf{t}_c$ is constructed as $\mathbf{t}_c = \hat{z} \times \text{proj}_{xy}(\bar{\mathbf{x}}_c) / \|\hat{z} \times \text{proj}_{xy}(\bar{\mathbf{x}}_c)\|$ where $\bar{\mathbf{x}}_c$ is the cell centroid. $\mathbf{R}_c$ is the axis-angle rotation by r_c about $\mathbf{t}_c$.

The rotation field $\{r_c\}$ is initialized from overhang geometry and the path-length gradient, then smoothed by solving a graph-Laplacian regularization problem. The deformation stage computes new vertex positions $\{v' \in \mathcal{V}'\}$ such that each deformed cell's centered shape matches the rotation-transformed original. Finally, the G-code remapping stage applies barycentric interpolation to recover physical machine coordinates.

4 Algorithmic Optimizations

The underlying formal structure and its complexity analysis is as follows: Let $n := n_c, m$ denote the number of directed cell-cell edges, n_g the number of G-code toolpath points after segmentation, and K the maximum number of neighbors any cell has in a given adjacency (face, edge, or vertex).

4.1 Sparse COO Scatter for Neighbor Adjacency

The reference implementation queries `input_tet.cell_neighbors(i, type)` inside a Python loop over cells and neighbor types, which dominates startup wall time for moderately large meshes. It then performs two additional $O(m)$ Python passes, one to build a dictionary of neighbor lists and one to pack those lists into a padded array required by downstream vectorized stages.

We reformulate adjacency construction as a sparse matrix product. Define the binary cell-point incidence matrix $\mathbf{C}_P \in \{0, 1\}^{n_c \times n_p}$ by

$$(\mathbf{C}_P)_{c,p} = \begin{cases} 1 & \text{if } p \in c, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

Then the symmetric cell-cell adjacency matrix

$$\mathbf{A} = \mathbf{C}_P \mathbf{C}_P^T \in \mathbb{N}^{n_c \times n_c} \quad (2)$$

has entries A_{ij} equal to the number of vertices shared by cells i and j . Off-diagonal entries take values in $\{1, 2, 3\}$, corresponding to *point*-, *edge*-, and *face-neighbors*, respectively.

From the COO representation of $\mathbf{A}$ we construct the padded edge-neighbor array $\mathbf{P} \in \mathbb{N}^{n_c \times K}$ (where $K = \max_c |\mathcal{N}_e(c)| + 1$) in three vectorized steps:

- (1) **Directed-edge concatenation.** Stack source indices $\mathbf{s} = [\mathbf{r}; \mathbf{c}; \mathbf{1}_{n_c}]$ and destination indices $\mathbf{d} = [\mathbf{c}; \mathbf{r}; \mathbf{1}_{n_c}]$, producing $2m + n_c$ directed edges including self-loops.
- (2) **Stable argsort by source.** $\sigma = \text{argsort}(\mathbf{s})$ in $O((m+n) \log(m+n))$ time via LAPACK-backed NumPy sort.
- (3) **Column-offset scatter.** Compute per-source degree counts $\ell = \text{bincount}(\mathbf{s}_\sigma)$, their cumulative sum $\mathbf{L}$, and define column indices $\mathbf{k}_i = i - L_{s_{\sigma_i}}$. Writing $\mathbf{P}_{s_{\sigma_i}, k_i} \leftarrow d_{\sigma_i}$ fills the padded array in a single NumPy advanced-indexing operation.

The resulting procedure has total complexity $O(m \log m + n_c K)$ with no Python iteration inside the loop body, while providing a single array that all subsequent vectorized stages can consume without rebuild.

4.2 Surface Geodesic Distance via the Heat Method

The reference implementation computed graph distances $\{d_c\}$ over the tetrahedral dual graph via `scipy.sparse.csgraph.dijkstra`, then traced each cell's predecessor chain to the nearest bottom cell using a per-cell Python while-loop. This approach couples the distance field to the interior tetrahedralization, producing asymmetric and mesh-topology-dependent gradients that vary with TetGen's partitioning of the interior.

S5_Slicer replaces this entirely with a surface-intrinsic geodesic distance field computed by the heat method [18]. Let $= (\mathcal{V}_S,)$ be the triangulated boundary surface extracted from $\mathcal{M}$, with $V \in \mathbb{R}^{n_s \times 3}$ the surface vertex positions and $F \in \mathbb{N}^{n_f \times 3}$ the triangle index array. The bottom source set is

$$\mathcal{B}_S = \{v \in \mathcal{V}_S : V_{v,z} < \min_{u \in \mathcal{V}_S} V_{u,z} + \delta\}, \quad (3)$$

where $\delta = 0.3$ mm is a threshold that selects the bottommost surface vertices independent of any tetrahedral cell membership. A `potpourri3d.MeshHeatMethodDistanceSolver` is constructed once on $(\mathbf{V}, \mathbf{F})$ and queried via `compute_distance_multisource( $\mathcal{B}_S$ )`, yielding a smooth, surface-intrinsic scalar field $u : \mathcal{V}_S \rightarrow \mathbb{R}_{\geq 0}$ that is independent of the interior tetrahedralization and free of the negative values and asymmetries produced by graph-distance approximations on coarse dual graphs.

The vertex distances $\mathbf{u} \in \mathbb{R}^{n_s}$ are then averaged onto tet cells via `np.add.at` scatter over the `vtkOriginalCellIds` face-to-cell map, with cells having no surface face assigned $d_c = \text{NaN}$. Cells whose d_c falls below the 15th percentile of the surface distance distribution are masked to zero rotation, as the heat gradient is unreliable within the source neighborhood.

4.3 Per-Face Gradient via the libigl Gradient Operator

The original path-length gradient was estimated per cell by fitting a plane via SVD to the Dijkstra distance values of edge-neighboring cells in $\mathbb{R}^3$. This neighborhood plane-fit amplifies discretization noise from the graph distance field and is sensitive to irregular tetrahedral partitioning near concavities and thin features, producing the checkerboard sign patterns and asymmetric rotation fields observed in `S4_Slicer`.

`S5_Slicer` replaces the SVD plane-fit with the standard per-face gradient operator from `libigl` [19]. For a scalar field $\mathbf{u} \in \mathbb{R}^{n_s}$ defined on surface vertices, the gradient operator $\mathbf{G} \in \mathbb{R}^{3n_f \times n_s}$ assembles one $3 \times n_s$ block per triangle using cotangent weights derived from the triangle geometry alone:

$$\mathbf{G} = \text{igl.grad}(\mathbf{V}, \mathbf{F}) \in \mathbb{R}^{3n_f \times n_s}, \quad (4)$$

so that $\mathbf{G}\mathbf{u} \in \mathbb{R}^{3n_f}$ stacks the per-face gradient vectors. Reshaping yields $\mathbf{g} \in \mathbb{R}^{n_f \times 3}$, one unit-normalized gradient vector per surface triangle. Because $\mathbf{G}$ depends only on the surface geometry and not on the tetrahedral interior, the resulting gradient field is surface-intrinsic and free of tetrahedralization artifacts.

The per-face gradients are normalized to unit length before scatter-averaging onto tet cells:

$$\hat{\mathbf{g}}_f = \frac{\mathbf{g}_f}{\|\mathbf{g}_f\| + \epsilon}, \quad \epsilon = 10^{-8}. \quad (5)$$

After the `np.add.at` scatter over `vtkOriginalCellIds`, the averaged cell gradients are renormalized, since averaging unit vectors reduces their magnitude. The radial rotation sign is then recovered as

$$s_c = -\text{sign}(\hat{\mathbf{e}}_{r,c} \cdot \hat{\mathbf{g}}_{c,xy}), \quad (6)$$

where $\hat{\mathbf{e}}_{r,c}$ is the normalized radial direction of cell c in the xy -plane and $\hat{\mathbf{g}}_{c,xy}$ is the projection of the cell gradient onto xy . The negation reflects the convention that the geodesic gradient points away from the base (increasing d_c), while a positive rotation corrects overhangs back toward the base. Cells near the source boundary (low d_c) and interior cells (NaN gradient) are zeroed before the rotation field is assembled.

The complexity of this stage drops from $O(nK^3)$ per-cell SVD in Python to a single sparse matrix-vector product $\mathbf{G}\mathbf{u}$ of cost $O(n_f)$, followed by $O(n_f)$ normalization and $O(n_f)$ scatter, with no Python iteration.

4.4 Direct Sparse Solve for Rotation-Field Smoothing

The rotation field $\mathbf{r} = (r_1, \dots, r_{n_c})$ is first initialized from overhang angles and path-length gradients. Raw initialization produces a spatially erratic field; smoothing is required. The original objective is

$$\min_{\mathbf{r}} W \sum_{(i,j) \in \mathcal{E}_F} (r_i - r_j)^2 + \sum_{i \in \mathcal{V}_{\text{init}}} (r_i - r_i^{\text{init}})^2, \quad (7)$$

where $\mathcal{E}_F$ is the set of face-neighbor pairs, $W > 0$ is a smoothing weight, and $\mathcal{V}_{\text{init}} = \{i : r_i^{\text{init}} \text{ is finite}\}$. This is a *strictly convex quadratic* in $\mathbf{r}$.

Derivation. Let $\mathbf{B} \in \{-1, 0, 1\}^{|\mathcal{E}_F| \times n_c}$ be the oriented incidence matrix of the face-neighbor graph. Then $\sum_{(i,j) \in \mathcal{E}_F} (r_i - r_j)^2 = \mathbf{r}^T \mathbf{B}^T \mathbf{B} \mathbf{r} = \mathbf{r}^T \mathcal{L}_F \mathbf{r}$, where $\mathcal{L}_F = \mathbf{D}_F - \mathbf{A}_F$ is the graph Laplacian of $(\mathcal{C}, \mathcal{E}_F)$ and equals $\mathbf{B}^T \mathbf{B}$. Let $\mathbf{M} = \text{diag}(\mathbb{I}[i \in \mathcal{V}_{\text{init}}])$ denote the anchor mask. Equation (7) becomes

$$\min_{\mathbf{r}} W \mathbf{r}^T \mathcal{L}_F \mathbf{r} + (\mathbf{r} - \mathbf{r}^{\text{init}})^T \mathbf{M} (\mathbf{r} - \mathbf{r}^{\text{init}}). \quad (8)$$

Setting the gradient to zero yields the symmetric linear system

$$(W \mathcal{L}_F + \mathbf{M} + \epsilon \mathbf{I}) \mathbf{r} = \mathbf{M} \mathbf{r}^{\text{init}}, \quad (9)$$

where $\epsilon > 0$ is a tiny ridge term ($\epsilon = 10^{-10}$) added to guarantee strict positive-definiteness when $\mathbf{M}$ is rank-deficient. Without $\epsilon \mathbf{I}$, the Laplacian has a one-dimensional null space (the constant vector) on each connected component of $\mathcal{E}_F$, so when $\mathbf{M}$ fails to anchor a component, UMFPACK reports a singular matrix. The ϵ term anchors disconnected components to zero at negligible distortion.

`S4_Slicer` did not recognize this structure and instead passed the residual vector to `scipy.optimize.least_squares` with `method='trf'`, which performs up to $T \leq 100$ trust-region iterations, each requiring a full Jacobian COO $\rightarrow$ CSR rebuild of the same $O(\text{nnz})$ matrix. `S5_Slicer` assembles the symmetric sparse system (9) *once* and calls `scipy.sparse.linalg.spsolve`, which dispatches to UMFPACK for an LU factorization with AMD ordering.

The complexity drops from $O(T \cdot \text{nnz})$ to $O(\text{nnz}^{1.5})$ for the LU factorization, with prefactor reductions of two to three orders of magnitude in practice because $T \gg 1$ and because the original TRF solver paid repeated Jacobian-assembly cost.

4.5 Parallel Sparse LSQR for Mesh Deformation

The deformation stage finds new vertex positions $\mathbf{V}' \in \mathbb{R}^{n_p \times 3}$ such that for each cell c , the centered local shape after applying the rotation $\mathbf{R}_c$ matches the centered shape of the deformed mesh. Let $\mathbf{N} \in \mathbb{R}^{4 \times 4}$ be the centering operator $\mathbf{N} = \mathbf{I}_4 - \frac{1}{4} \mathbf{1}_4 \mathbf{1}_4^T$, so that $\mathbf{N}\mathbf{v}_c$ centers the four vertices of cell c about their centroid.

The deformation objective is

$$\min_{\mathbf{V}'} \sum_{c \in \mathcal{C}} \|\mathbf{R}_c \mathbf{N} \mathbf{v}_c - \mathbf{N} \mathbf{v}'_c\|_F^2, \quad (10)$$

where $\mathbf{v}_c, \mathbf{v}'_c \in \mathbb{R}^{4 \times 3}$ are the per-cell vertex stacks in the undeformed and deformed configurations. This is quadratic in the deformed vertex coordinates.

Decomposition. The Frobenius norm decomposes into three independent coordinate-wise sums,

$$\sum_c \|\mathbf{R}_c \mathbf{N} \mathbf{v}_c - \mathbf{N} \mathbf{v}'_c\|_F^2 = \sum_{j \in \{x, y, z\}} \sum_c \left\| (\mathbf{R}_c \mathbf{N} \mathbf{v}_c)_{:,j} - \mathbf{N}(\mathbf{v}'_c)_{:,j} \right\|_2^2, \quad (11)$$

so the full problem separates into three independent sparse linear least-squares subproblems:

$$\min_{\mathbf{v}'_{(j)} \in \mathbb{R}^{n_p}} \|\mathbf{A} \mathbf{v}'_{(j)} - \mathbf{b}_{(j)}\|_2^2, \quad j \in \{x, y, z\}. \quad (12)$$

The matrix $\mathbf{A} \in \mathbb{R}^{4n_c \times n_p}$ is the cell-local centering operator, block-indexed by cell and local vertex. For cell c with vertex indices $(p_0, p_1, p_2, p_3) = \text{cells}[c]$, the $(4c + i)$ -th row of $\mathbf{A}$ has four non-zeros,

$$A_{4c+i, p_k} = N_{i,k} = \begin{cases} \frac{3}{4} & i = k, \\ -\frac{1}{4} & i \neq k. \end{cases} \quad (13)$$

Crucially, $\mathbf{A}$ depends only on the mesh topology and the centering matrix $\mathbf{N}$ and *never changes across deformation iterations*. The right-hand side $\mathbf{b}_{(j)} \in \mathbb{R}^{4n_c}$ is derived from $\mathbf{R}_c \mathbf{N} \mathbf{v}_c$ via a single einsum.

Formulation bug in S4_Slicer. The original S4_Slicer formulation passed *per-cell squared Frobenius norms* as the residuals to `least_squares` (TRF squares them internally), which means TRF actually minimizes

$$\sum_c (\|\mathbf{M}_c\|_F^2)^2 = \sum_c \|\mathbf{M}_c\|_F^4, \quad (14)$$

where $\mathbf{M}_c = \mathbf{R}_c \mathbf{N} \mathbf{v}_c - \mathbf{N} \mathbf{v}'_c$. This quartic objective shares the same global minimum as the intended quadratic (both are zero when every $\mathbf{M}_c = 0$), but its Hessian vanishes at the minimizer, dramatically flattening the curvature TRF sees and slowing convergence by an order of magnitude. Returning the *linear* residuals $\mathbf{A} \mathbf{v}'_{(j)} - \mathbf{b}_{(j)}$ restores the correct quadratic objective.

Parallel LSQR with warm start. We solve (12) using `scipy.sparse.linalg.lsqr`, a Golub–Kahan bidiagonalization-based iterative solver well suited to sparse rectangular systems. Three properties of the solver and our formulation combine to yield a $\sim 9\times$ wall-time reduction:

- (1) $\mathbf{A}$ is assembled *once*, outside any iteration.
- (2) LSQR is warm-started at $\mathbf{x}_0^{(j)} = \mathbf{V}_{:,j}$, the current vertex positions. Because the undeformed mesh is not far from the deformed one, this reduces the iteration count from the worst-case $O(n_p^{1/2})$ to typically fewer than 30 for well-conditioned meshes.
- (3) The three coordinate subsystems are fully independent and are dispatched to a `ThreadPoolExecutor` with `max_workers=3`. SciPy's CSR matvec releases the Python GIL, so the three threads achieve true parallel execution and deliver a near-ideal $3\times$ speedup on the LSQR phase.

4.6 Batched Barycentric Coordinate Computation

G-code reformation requires, for each toolpath point $\mathbf{p}_k \in \mathbb{R}^3$, the barycentric coordinates $\lambda_k = (\lambda_{k,1}, \lambda_{k,2}, \lambda_{k,3}, \lambda_{k,4})$ of $\mathbf{p}_k$ within its containing tetrahedron. Let $(\mathbf{v}_a, \mathbf{v}_b, \mathbf{v}_c, \mathbf{v}_d)$ be the four vertices of

the containing cell. The first three coordinates satisfy the 3×3 linear system

$$\mathbf{T}_k \lambda_{k,1:3} = \mathbf{p}_k - \mathbf{v}_d, \quad \mathbf{T}_k = [\mathbf{v}_a - \mathbf{v}_d \mid \mathbf{v}_b - \mathbf{v}_d \mid \mathbf{v}_c - \mathbf{v}_d], \quad (15)$$

with $\lambda_{k,4} = 1 - \sum_{j=1}^3 \lambda_{k,j}$. S4_Slicer calls `np.linalg.solve` once per toolpath point inside a sequential Python loop, incurring n_g Python-to-C dispatches and preventing any LAPACK batching.

The full operation is reformulated as a single batched gufunc. Stack all $\mathbf{T}_k$ into a $(n_v, 3, 3)$ tensor where $n_v \leq n_g$ is the number of points with a valid containing cell, and all right-hand sides into a $(n_v, 3, 1)$ tensor. A single call to `np.linalg.solve` with gufunc signature $(m, m), (m, n) \rightarrow (m, n)$ dispatches one LAPACK `gesv` call across the entire batch; the trailing $n = 1$ column dimension is essential, otherwise NumPy interprets the batch axis as the core m -dimension and raises a shape-mismatch error.

Cells with singular $\mathbf{T}_k$ (degenerate tetrahedra) are handled by a try/except fallback to per-element solves. The validity check $\sum_j \lambda_{k,j} \leq 1.01$ is evaluated as a vectorized inequality.

Pre-computed positions and rotations are then stored in arrays $\tilde{\mathbf{P}} \in \mathbb{R}^{n_g \times 3}$ and $\tilde{\mathbf{r}} \in \mathbb{R}^{n_g}$ for $O(1)$ lookup during the subsequent G-code-writing loop, eliminating all per-point linear-algebra work from the sequential phase.

4.7 Batch SVD for Vertex Rotation Recovery

To interpolate nozzle rotations smoothly across toolpath segments, S5_Slicer recovers a signed planar rotation angle per cell from the deformation itself, then scatter-averages the angles onto vertices. Let $\mathbf{U}_c \in \mathbb{R}^{4 \times 3}$ and $\mathbf{U}'_c \in \mathbb{R}^{4 \times 3}$ be the centered vertex stacks of cell c in the undeformed and deformed configurations, each projected onto the local 2-D frame $(\hat{\mathbf{e}}_{r,c}, \hat{\mathbf{z}})$:

$$U_{c,i,1} = \hat{\mathbf{e}}_{r,c} \cdot (\mathbf{v}_{c,i} - \bar{\mathbf{x}}_c), \quad U_{c,i,2} = (\mathbf{v}_{c,i} - \bar{\mathbf{x}}_c) \cdot \hat{\mathbf{z}}. \quad (16)$$

The best-fit rotation from $\mathbf{U}_c$ to $\mathbf{U}'_c$ is the orthogonal Procrustes problem

$$\mathbf{R}_c^* = \arg \min_{\mathbf{R} \in \text{SO}(2)} \|\mathbf{U}'_c - \mathbf{U}_c \mathbf{R}^T\|_F^2, \quad (17)$$

with closed-form solution from the SVD of the cross-covariance [14]:

$$\mathbf{H}_c = \mathbf{U}'_c{}^T \mathbf{U}_c, \quad \mathbf{H}_c = \mathbf{U}_c^{\text{svd}} \Sigma_c \mathbf{V}_c^T, \quad \mathbf{R}_c^* = \mathbf{U}_c^{\text{svd}} \mathbf{V}_c^T. \quad (18)$$

The signed angle is then $\phi_c = \text{atan2}(R_{c,1,0}^*, R_{c,0,0}^*)$, clipped to the admissible B -axis range.

To batch the entire pipeline: (i) one `einsum('ni,nki->nk')` projection per dimension; (ii) one `einsum('nki,nkj->nij')` cross-covariance over all cells; (iii) one `np.linalg.svd` on the $(n_c, 2, 2)$ tensor; and (iv) one vectorized `arccos` with sign correction from the $R_{c,1,0}^*$ entry. A final `np.add.at` scatter-reduces cell angles onto vertices weighted by the inverse per-vertex incidence count $1/|\{c : v \in c\}|$, which is itself pre-computed by `np.bincount`.

4.8 Vectorized Tetrahedral Volume Scaling

Each deformed cell acquires a volume ratio $\zeta_c = \text{vol}(c_{\text{orig}})/\text{vol}(c_{\text{def}})$ used to compensate extrusion volume in the final G-code. The signed volume of a tetrahedron with vertices $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3 \in \mathbb{R}^3$ is

$$\text{vol}(c) = \frac{1}{6} |\det[\mathbf{v}_1 - \mathbf{v}_0, \mathbf{v}_2 - \mathbf{v}_0, \mathbf{v}_3 - \mathbf{v}_0]|. \quad (19)$$

We stack the edge-vector matrix $E_c = [v_1 - v_0, v_2 - v_0, v_3 - v_0]$ into a tensor $E \in \mathbb{R}^{n_c \times 3 \times 3}$ for both the deformed and undeformed configurations, then compute all volumes via the single call `np.abs(np.linalg.det(E)) / 6.0`, where `det` broadcasts over the leading batch axis through its `gufunc` signature $(m, m) \rightarrow ()$.

4.9 Constant-Time Validity Pre-computation in G-code Reformation

The inner G-code reformation loop examines each of the n_g toolpath points and checks whether its pre-computed new position contains a NaN coordinate. The original pattern `np.any(np.isnan(pos))` allocates a small boolean array on every iteration. We pre-compute the validity vector

$$v = \neg(\text{np.any}(\text{np.isnan}(\tilde{P}), \text{axis}=1)) \in \{0, 1\}^{n_g} \quad (20)$$

once outside the loop in a single vectorized pass, and replace the per-iteration NaN check with an $O(1)$ array lookup v_k . Although the per-iteration savings are small, over $n_g \approx 2 \times 10^5$ iterations the elimination of ephemeral allocation removes a measurable fraction of the reformation stage's wall time.

5 Complexity Summary

Table 1 summarizes the asymptotic complexity of each pipeline stage before and after optimization. Let $n = n_c$, m the number of directed cell-cell edges, n_g the number of G-code toolpath points, T the number of TRF outer iterations in the reference implementation, and D the depth of the Dijkstra predecessor tree.

The two largest absolute wall-time improvements are the rotation-field smoothing (§4.4) and the mesh deformation (§4.5). TRF ran up to $T = 100$ Jacobian evaluations for smoothing, each requiring a COO-to-CSR rebuild of the same $O(m)$ sparse matrix; `spsolve` reduces this to a single LU factorization. For deformation, the quartic TRF formulation ran up to $T = 1000$ function evaluations; parallel LSQR with warm starts converges in fewer than 30 iterations per spatial dimension.

6 Implementation Notes

6.1 Numerical Stability

The diagonal regularization $\epsilon = 10^{-10}$ added to the rotation-field system (9) ensures strict positive-definiteness when M fails to anchor some connected component of the face-neighbor graph. Without this term, the Laplacian contributes a null space spanned by the constant vector on each disconnected component, and UMFPACK reports a singular matrix. The ϵ term anchors disconnected components to zero rotation at distortion bounded by $\epsilon \cdot \|r^{\text{init}}\|_\infty / \lambda_{\min}(W\mathcal{L}_F)$, which is negligible.

In the deformation solve (§4.5), warm-start initialization $x_0^{(j)} = V_{:,j}$ reduces LSQR iterations from the default $O(n_p^{1/2})$ worst case to typically fewer than 30 for well-conditioned meshes, monitored via the Paige–Saunders tolerances `atol = btol = 10-7`.

6.2 Upgrade Paths for Large Models

For models with $n_c \gtrsim 5 \times 10^4$, the following solvers offer further reductions:

- **scikit-sparse CHOLMOD** [4] for direct Cholesky factorization of $A^T A$ in the deformation system. Since A has a regular sparsity pattern determined by the mesh, CHOLMOD's supernodal factorization outperforms LSQR once $n_p \gtrsim 5 \times 10^4$.
- **PyPardiso** (Intel MKL PARDISO) is a drop-in replacement for `spsolve` with multi-threaded LU factorization that yields 4–8× speedup over UMFPACK on the rotation-field system for multi-core hosts.
- **CuPy cuSPARSE**: GPU-accelerated sparse matrix-vector products permit 10–100× speedups on large-mesh LSQR once GPU transfer cost is amortized across sufficient iterations.

6.3 Memory Footprint

The padded neighbor array P of shape (n_c, K) uses `np.int32` storage; for a typical mesh with $K \approx 20$ and $n_c = 3 \times 10^4$, this requires ≈ 2.4 MB, easily in cache-adjacent memory. The $(n_v, 3, 3)$ tensor for batched barycentric solves requires 72 bytes per valid G-code point; for a 2×10^5 -point toolpath, peak allocation is ≈ 14 MB. All intermediate tensors are released promptly through Python's reference-counting GC.

7 Discussion

The most conceptually significant optimization is the recognition in Section 4.4 that Laplacian smoothing is a quadratic program. This fact is well established in geometry processing [12; 13] but was not exploited in the reference implementation, which dispatched the problem to a general-purpose nonlinear solver. Identifying the latent structure alone reduced rotation-field computation for moderately complex models from tens of seconds to < 0.1 s. The lesson generalizes: *any objective expressible as a sum of squared affine functions of the decision variables is quadratic, and its minimizer is found in one shot by assembling and solving a sparse linear system*. Profiling identifies hot paths; mathematical characterization—linear, quadratic, or genuinely nonlinear—is what unlocks the order-of-magnitude speedup.

A subtler but equally instructive issue is the formulation bug in the deformation stage (Section 4.5). Returning per-cell $\|M_c\|_F^2$ as residuals to a least-squares solver causes TRF to minimize $\sum_c \|M_c\|_F^4$, a quartic surrogate that shares the same global minimizer as the quadratic but has a degenerate (zero) Hessian at that minimizer. The solver still converges, but the distorted local curvature slows progress by roughly an order of magnitude. Recognizing and correcting the formulation error was as important as the subsequent algorithmic substitution.

The parallel LSQR solve exploits a property of SciPy's sparse routines that is often overlooked in pure-Python workflows: CSR matrix-vector products release the GIL. Python threads, normally constrained to sequential execution by the GIL, thus achieve true parallelism for LSQR's inner loop. Because the three coordinate dimensions are fully independent, thread-level parallelism delivers a near-ideal 3× speedup on the LSQR phase without resort to multiprocessing or external compilation.

From an optimization-theoretic perspective, the rotation-field smoothing problem is a quadratic program on a graph: minimize a weighted sum of squared differences between adjacent decision

Table 1. Asymptotic complexity of pipeline stages in S4_Slicer versus S5_Slicer.

Stage	S4_Slicer (original)	S5_Slicer (this work)	Primary technique
Neighbor adjacency (§4.1)	$O(n)$ Python calls + $O(m)$ Python passes	$O(m \log m)$ NumPy	COO argsort + cumsum scatter
Geodesic distance field (§4.2)	$O(nD \mathcal{B})$ Python while-loop over Dijkstra predecessors	$O(n_s)$ heat method on surface	potpourri3d multisource heat solve
Surface gradient (§4.3)	$O(nK^3)$ per-cell SVD plane-fit in Python	$O(n_f)$ sparse matvec	igl.grad cotangent operator
Rotation-field smoothing (§4.4)	$O(T \cdot \text{nnz})$ TRF on quadratic	$O(\text{nnz}^{1.5})$ UMFPACK LU	Laplacian QP $\rightarrow$ spsolve
Mesh deformation (§4.5)	$O(T \cdot n^2)$ TRF on quartic	$O(T_{\text{LSQR}} \cdot \text{nnz})$ parallel LSQR	Linear LS $\rightarrow$ 3-thread LSQR
Barycentric transforms (§4.6)	n_g sequential solve calls	1 batched solve gufunc	np.linalg.solve batched
Vertex rotation recovery (§4.7)	$O(n)$ per-cell SVD in Python	$O(n)$ batched SVD	einsum + batched svd
Volume scaling (§4.8)	$O(n)$ per-cell det in Python	$O(n)$ batched det	np.linalg.det broadcast
Validity masking (§4.9)	n_g ephemeral allocations	1 pre-computed mask	Vectorized pre-pass

variables, subject to anchor terms that penalize deviation from target values. This is precisely the structure of network-based smoothing problems [1], and the same sparse-direct solution techniques apply.

8 Conclusion

This paper presented *S5_Slicer*, a collection of nine algorithmic optimizations applied to a nonplanar FDM slicer for rotational 4-axis printers. The central theme is the identification of latent linear and quadratic structure in computations previously handled by general-purpose iterative solvers or Python-level loops. By leveraging sparse direct solvers (UMFPACK LU via spsolve), batched NumPy gufuncs (svd, solve, det), vectorized graph operations (COO scatter, fixed-point mask propagation), and thread-level parallelism with GIL-releasing CSR matvec, we reduced the dominant stages from multi-second or multi-minute operations to sub-second routines on commodity hardware, a four-order-of-magnitude wall-time improvement for a representative 50,000-cell mesh.

The broader principle is that numerically intensive geometry-processing pipelines frequently contain hidden linear or quadratic structure. Profiling to identify hot paths, then mathematically characterizing whether those paths are linear, quadratic, or genuinely nonlinear, is the key step that unlocks order-of-magnitude improvements. The upgrade paths identified (CHOLMOD, PyPardiso, CuPy) indicate further scaling to larger models is feasible with existing open-source infrastructure. We conjecture that the same reformulation strategy applied here—from TRF to direct sparse solves and batched gufuncs—generalizes to other multi-axis slicing frameworks and, more broadly, to any geometry-processing pipeline whose hot paths admit affine residuals.

References

- [1] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [2] C. Bellehumeur, L. Gibson, Q. Sun, and P. Gu. Modeling of bond formation between polymer filaments in the fused deposition modeling process. *J. Manufacturing Processes*, 6(2):170–178, 2004.
- [3] J. Bird. *S4_Slicer: An open-source nonplanar slicer for 4-axis 3D printers*. https://github.com/jyjbld/S4_Slicer, 2024.
- [4] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Trans. Math. Software*, 35(3):1–14, 2008.
- [5] T. A. Davis and I. S. Duff. An unsymmetric-pattern multifrontal method for sparse LU factorization. *SIAM J. Matrix Anal. Appl.*, 18(1):140–158, 1997.
- [6] J. Etienne, N. Ray, D. Panozzo, S. Hornus, C. Wang, J. Martinez, S. McMains, M. Alexa, B. Wyvill, and S. Lefebvre. CurviSlicer: Slightly curved slicing for 3-axis printers. *ACM Trans. Graph.*, 38(4):1–11, 2019.

- [7] G. Fang, T. Zhang, S. Zhong, X. Chen, Z. Zhong, and C. C. L. Wang. Reinforced FDM: Multi-axis filament alignment with controlled anisotropic strength. *ACM Trans. Graph.*, 39(6):1–15, 2020.
- [8] M. S. Floater. Mean value coordinates. *Computer Aided Geometric Design*, 20(1):19–27, 2003.
- [9] C. R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [10] Y. Ichida, R. Kondo, Y. Morimoto, and K. Yamamoto. Curved-layer fused deposition modeling for non-planar surface fabrication. In *Proc. ASPE Annual Meeting*, 2006.
- [11] H. Si. TetGen, a Delaunay-based quality tetrahedral mesh generator. *ACM Trans. Math. Software*, 41(2):1–36, 2015.
- [12] O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel. Laplacian surface editing. In *Proc. Symposium on Geometry Processing*, 2004.
- [13] G. Taubin. A signal processing approach to fair surface design. In *Proc. SIGGRAPH*, pp. 351–358, 1995.
- [14] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Trans. Pattern Anal. Mach. Intell.*, 13(4):376–380, 1991.
- [15] P. Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.
- [16] R. Wu, C. Peng, F. Isvoranu, and M. Pauly. Print-and-fold: Fabricating thin-shell objects. *ACM Trans. Graph.*, 36(4):1–9, 2017.
- [17] T. Zhang, G. Fang, Y. Huang, N. Dutta, S. Lefebvre, Z. M. Kilic, and C. C. L. Wang. Singular path planning for single-arm multi-axis additive manufacturing.
- [18] K. Crane, C. Weischedel, and M. Wardetzky. Geodesics in heat: A transfer operator approach to geodesics. *ACM Trans. Graph.*, 32(5):1–11, 2013.
- [19] A. Jacobson, D. Panozzo, et al. libigl: A simple C++ geometry processing library. <https://libigl.github.io>, 2018. *ACM Trans. Graph.*, 41(4):1–18, 2022.